Object-Oriented Programming

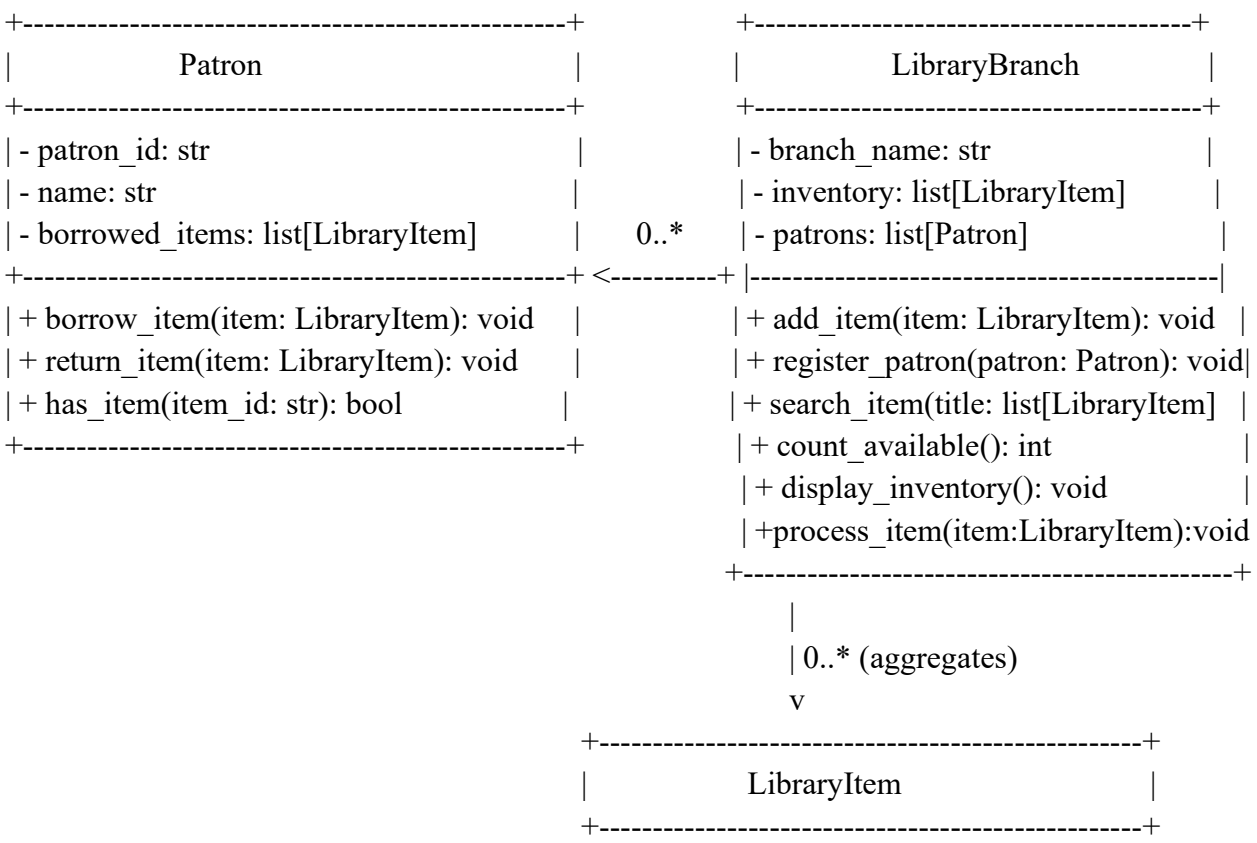
Assignment 1
Campus Digital Library System

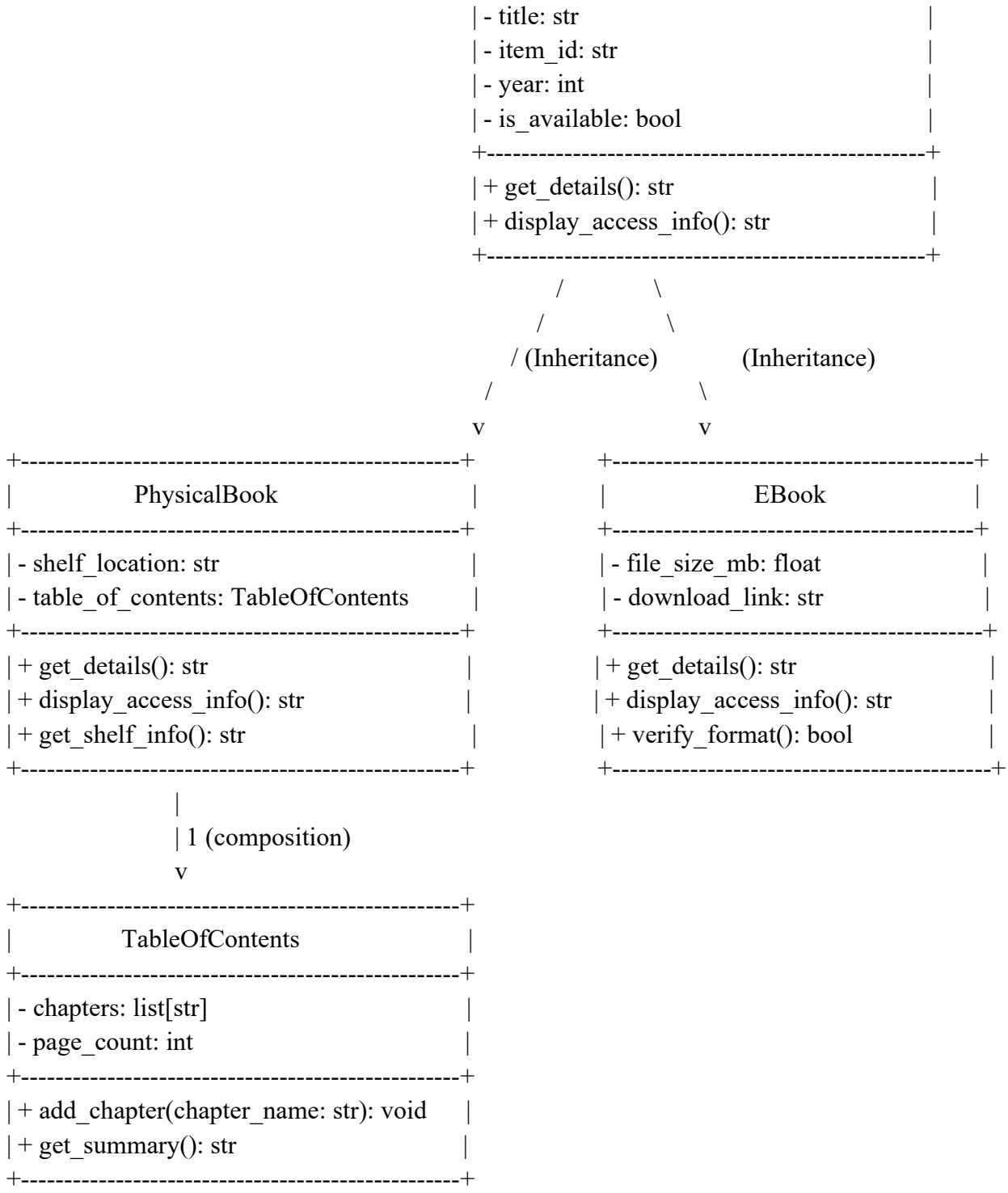
NAMES	STUDENT NUMBERS
Hamukoto Max JF	224164139
David Ali MT	224092677
Erastus Ndapewa O	222080833

1. Problem Statement

The University of Namibia wants a simple console-based system to manage its library resources. The system must handle both physical and digital books, manage patrons, and keep track of the items available at different library branches. The application should be designed using object-oriented principles. Specifically, it must demonstrate inheritance, composition and aggregation. The system must also include a method that uses polymorphism, a method that checks the specific type of an object, and at least one method that uses a basic algorithm such as searching or counting. A console application should be provided to test the main functionalities.

2. UML Class Diagram





Notation Legend:

• Inheritance (is-a): LibraryItem acts as superclass for PhysicalBook and EBook.

• Composition (part-of, strong ownership): PhysicalBook strongly owns TableOfContents.

• Aggregation (has-a, weak ownership): LibraryBranch aggregates LibraryItem and Patron collections.

3. Implementation (Python)

```
class TableOfContents:
    def __init__(self, page_count: int):
        self.page_count = page_count
        self.chapters = []

    def add_chapter(self, chapter_name: str):
        self.chapters.append(chapter_name)

    def get_summary(self) -> str:
        return f"Pages: {self.page_count}, Chapters: {len(self.chapters)}"

class LibraryItem:
    def __init__(self, title: str, item_id: str, year: int):
        self.title = title
        self.item_id = item_id
        self.year = year
        self.is_available = True

    def get_details(self) -> str:
        status = "Available" if self.is_available else "Checked Out"
        return f"[{self.item_id}] {self.title} ({self.year}) - {status}"

    def display_access_info(self):
        raise NotImplementedError("Must be implemented by subclasses")

class PhysicalBook(LibraryItem):
    def __init__(self, title: str, item_id: str, year: int, shelf_location: str, page_count: int):
        super().__init__(title, item_id, year)
        self.shelf_location = shelf_location
        self.table_of_contents = TableOfContents(page_count)

    def display_access_info(self) -> str:
        return f"Physical copy at shelf {self.shelf_location}."

    def get_shelf_info(self) -> str:
```

```
        return f"Shelf: {self.shelf_location}"

class EBook(LibraryItem):
    def __init__(self, title: str, item_id: str, year: int, file_size_mb: float, download_link: str):
        super().__init__(title, item_id, year)
        self.file_size_mb = file_size_mb
        self.download_link = download_link

    def display_access_info(self) -> str:
        return f"Digital copy. Download at {self.download_link}."

    def verify_format(self) -> bool:
        return self.download_link.endswith(".pdf")

class Patron:
    def __init__(self, patron_id: str, name: str):
        self.patron_id = patron_id
        self.name = name
        self.borrowed_items = []

    def borrow_item(self, item: LibraryItem):
        if item.is_available:
            item.is_available = False
            self.borrowed_items.append(item)
            print(f"{self.name} borrowed '{item.title}'.")
        else:
            print(f"'{item.title}' is not available.")

    def return_item(self, item: LibraryItem):
        if item in self.borrowed_items:
            self.borrowed_items.remove(item)
            item.is_available = True
            print(f"{self.name} returned '{item.title}'.")

    def has_item(self, item_id: str) -> bool:
        return any(item.item_id == item_id for item in self.borrowed_items)

class LibraryBranch:
    def __init__(self, branch_name: str):
        self.branch_name = branch_name
```

```
        self.inventory = []
        self.patrons = []

    def add_item(self, item: LibraryItem):
        self.inventory.append(item)

    def register_patron(self, patron: Patron):
        self.patrons.append(patron)

    def search_item(self, title: str) -> list:
        return [item for item in self.inventory if title.lower() in item.title.lower()]

    def count_available(self) -> int:
        return sum(1 for item in self.inventory if item.is_available)

    def display_inventory(self):
        print(f"Inventory at {self.branch_name}:")
        for item in self.inventory:
            print(f"  - {item.get_details()}")

    def process_item(self, item: LibraryItem):
        print(f"Item: {item.title}")
        print(f"Access info: {item.display_access_info()}")
        if isinstance(item, EBook):
            print(f"Type: EBook (Size: {item.file_size_mb} MB)")
        elif isinstance(item, PhysicalBook):
            print(f"Type: PhysicalBook (Shelf: {item.shelf_location})")

def main():
    library = LibraryBranch("Main Campus Library")

    book1 = PhysicalBook("Discrete Mathematics", "P001", 2023, "A-3", 450)
    book1.table_of_contents.add_chapter("Logic")
    book1.table_of_contents.add_chapter("Sets")
    ebook1 = EBook("Linear Algebra Guide", "E001", 2024, 12.5,
"https://library.unam.edu.na/linear.pdf")

    library.add_item(book1)
    library.add_item(ebook1)
```

```
    student = Patron("U001", "Max Hamukoto")
    library.register_patron(student)

    print("Search results for 'Linear':")
    results = library.search_item("Linear")
    for r in results:
        print(r.get_details())

    print(f"Available items: {library.count_available()}")

    print("Test polymorphism and type checking:")
    for item in library.inventory:
        library.process_item(item)

    print("Test borrowing book:")
    student.borrow_item(book1)
    print(f"Available items after borrowing: {library.count_available()}")

if __name__ == "__main__":
    main()
```

4. GitHub Repository

Repository link: https://github.com/maxjurgen77/campus-digital-library-oop.git